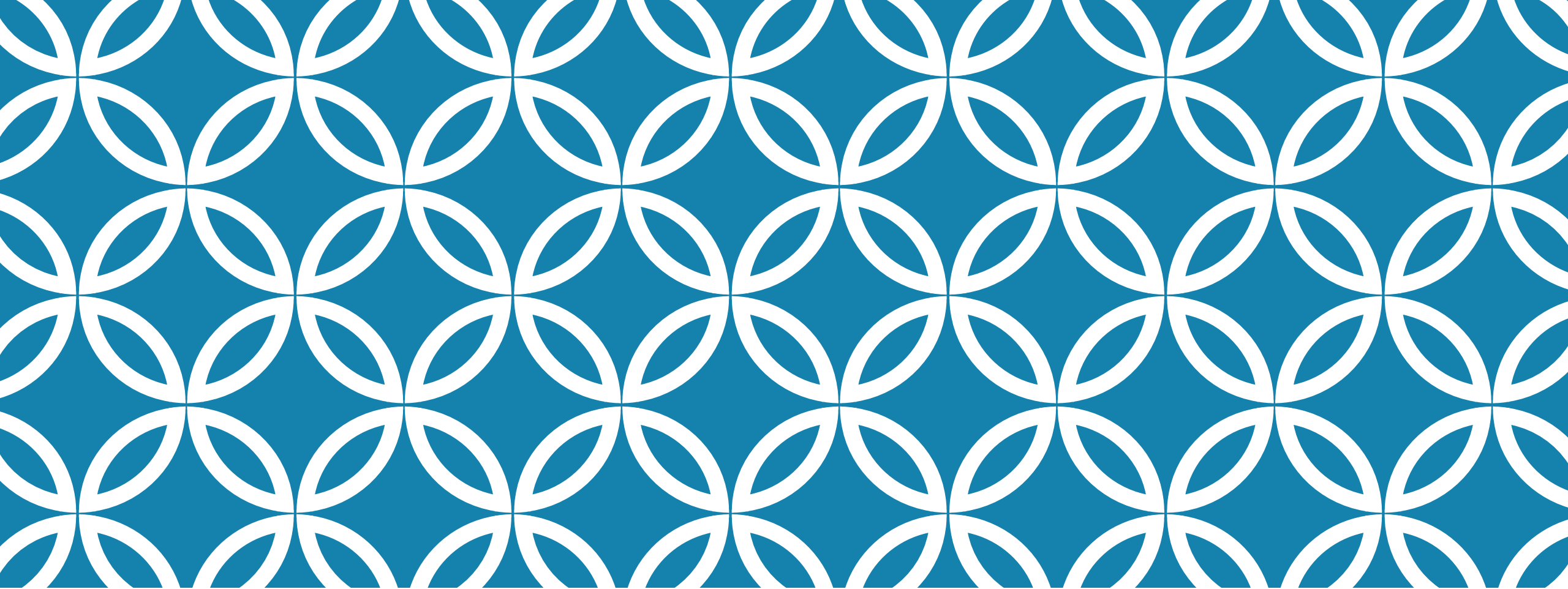




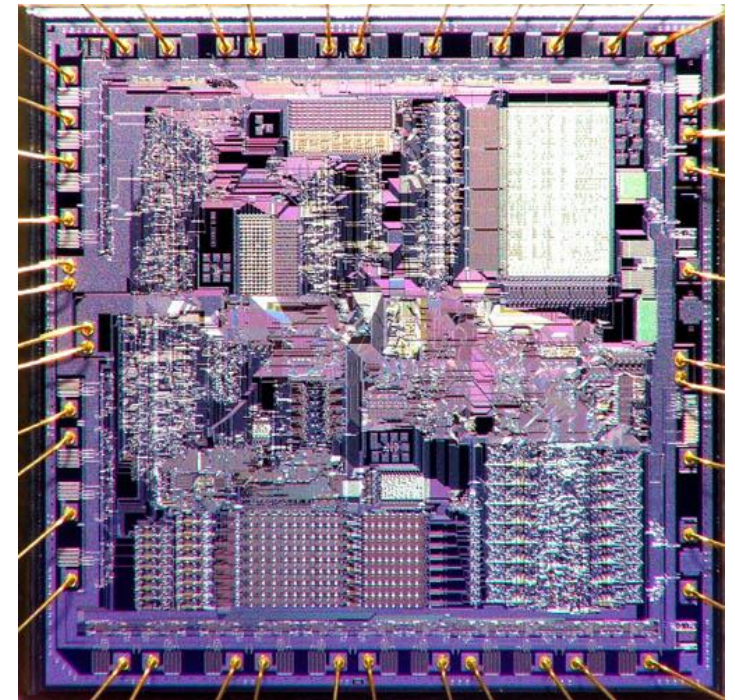
MICROPROCESSADOR i8086/8088

Arquitetura de Computadores

HISTÓRIA

Em 1978, Intel lança o primeiro processador de 16 bits: o 8086

- 117 instruções
- Podem processar bits, bytes, palavras e blocos de até 64Kbytes
- Barramento de endereços de 20 bits (memória até 1MB)
- Barramento de dados de 16 bits
- 64K portas de entrada/saída (I/O)
- 24 modos de endereçamento de operandos na memória
- 256 tipos de Interrupções
- Clock de 4,77 MHz
- Pastilha de CI de 40 pinos (29K transistores)



HISTÓRIA

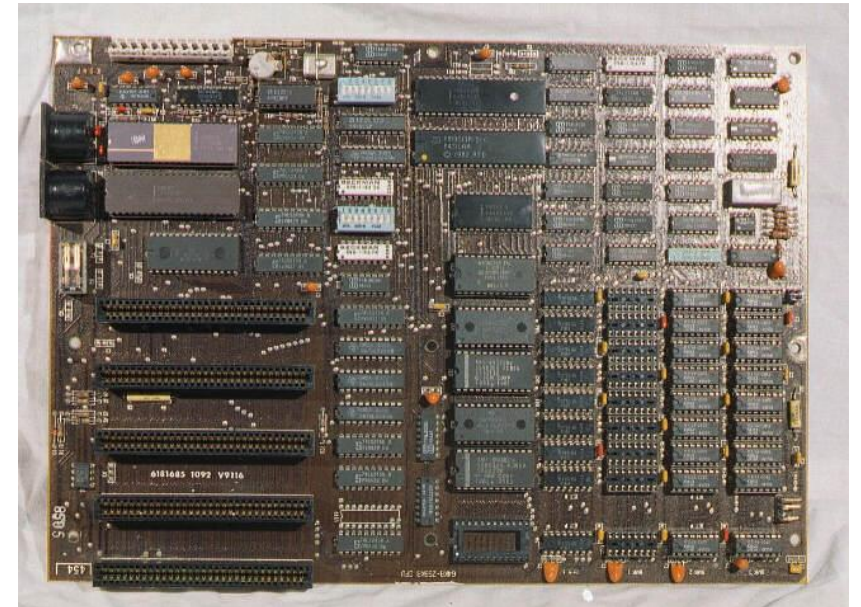
Em 1979, Intel lança um processador de menor custo: o 8088

- Barramento de dados de 8 bits



Em 1981, a IBM lança o primeiro microcomputador pessoal: IBM PC

- Microprocessador i8088 @ 4,77 MHz
- Memória de 64 Kbytes na placa-mãe (expansão até 256 Kbytes)
- 5 slots de expansão
- Drive de 5 1/4 " com capacidade de armazenamento de 360 Kbytes
- Custo de US\$ 1.565



HISTÓRIA

8086



			MAX MODE	MIN MODE
Vss (GND)	1	40	Vcc (5P)	
AD14	2	39	AD15	
AD13	3	38	A16/S3	
AD12	4	37	A17/S4	
AD11	5	36	A18/S5	
AD10	6	35	A19/S6	
AD9	7	34	BHE/S7	
AD8	8	33	MN/MX	
AD7	9	32	RD	
AD6	10	31	RQ/GT0	HOLD
AD5	11	30	RQ/GT1	HLDA
AD4	12	29	LOCK	WR
AD3	13	28	S2	M/IO
AD2	14	27	S1	DT/R
AD1	15	26	S0	DEN
AD0	16	25	QS0	ALE
NMI	17	24	QS1	INTA
INTR	18	23	TEST	
CLK	19	22	READY	
Vss (GND)	20	21	RESET	

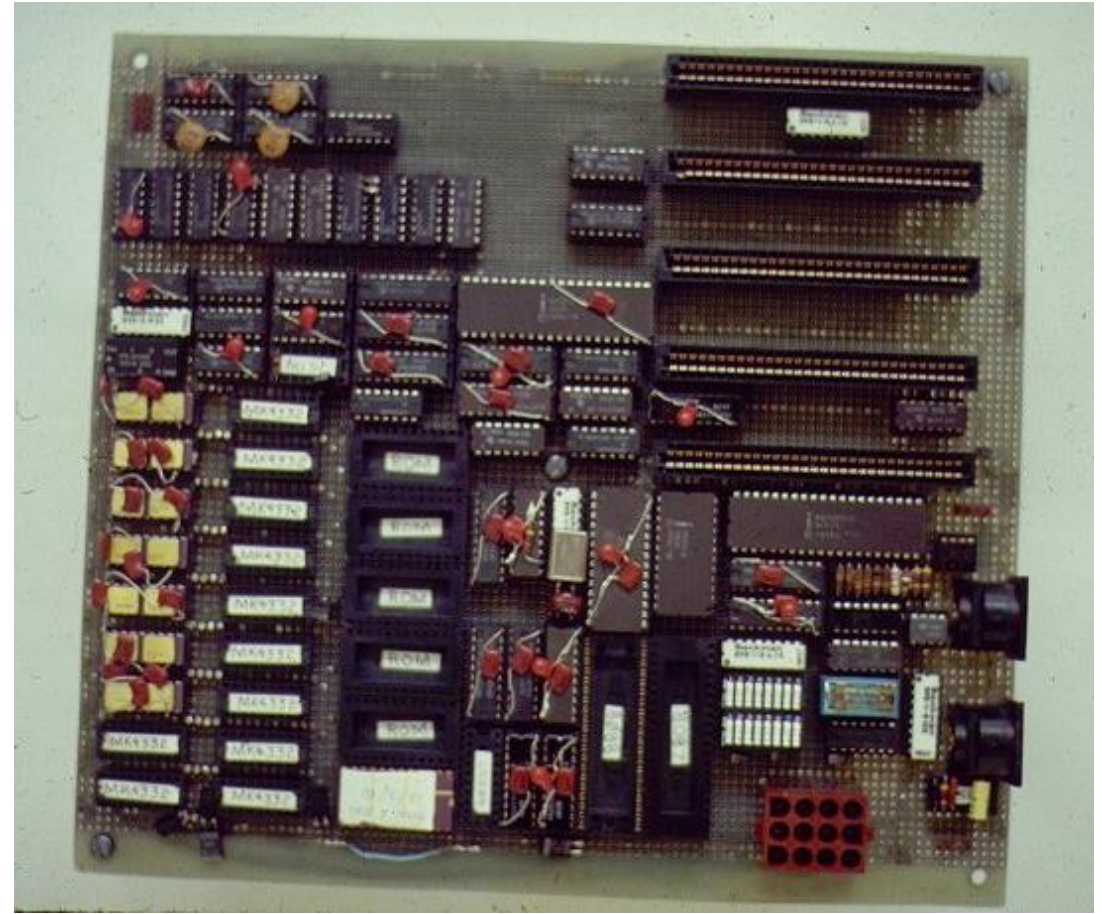
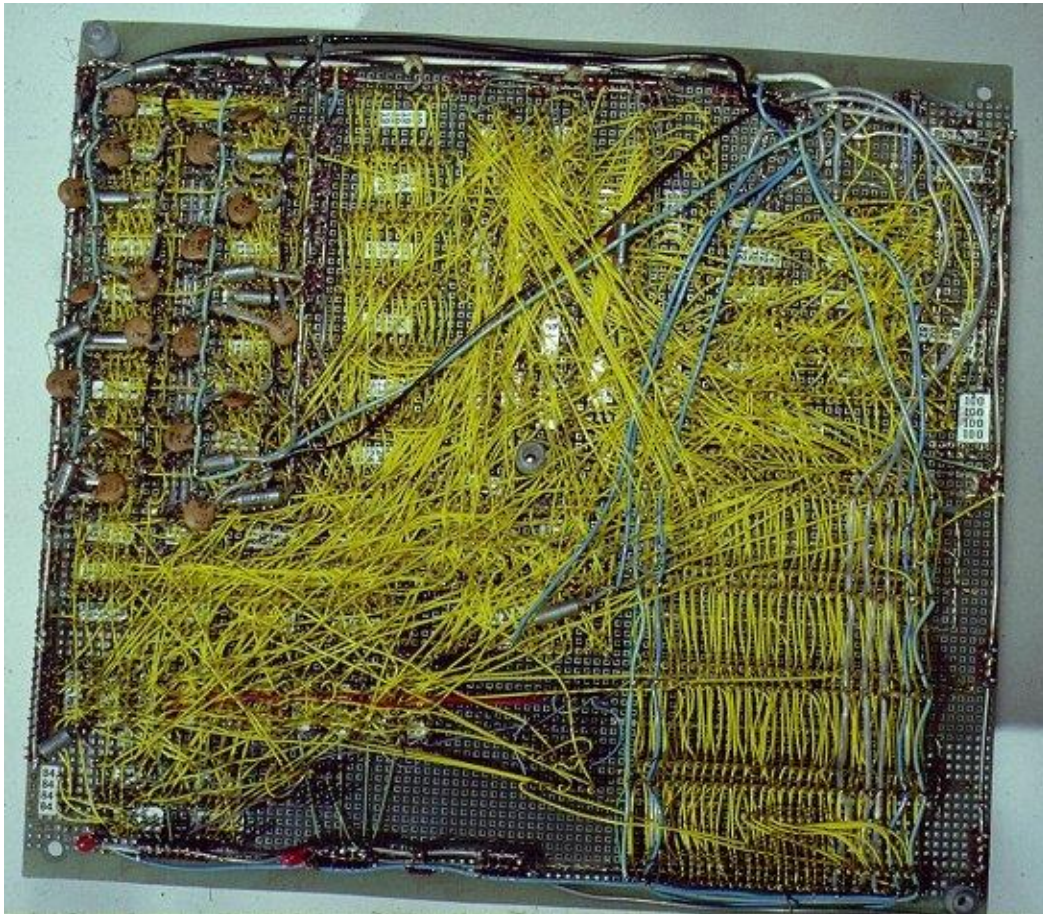
8088



GND	1	40	VCC
A14	2	39	A15
A13	3	38	A16/S3
A12	4	37	A17/S4
A11	5	36	A18/S5
A10	6	35	A19/S6
A9	7	34	SS0
A8	8	33	MN/MX
AD7	9	32	RD
AD6	10	31	RQ/GT0
AD5	11	30	RQ/GT1
AD4	12	29	LOCK
AD3	13	28	S2
AD2	14	27	S1
AD1	15	26	S0
AD0	16	25	QS0
NMI	17	24	QS1
INTR	18	23	TEST
CLK	19	22	READY
GND	20	21	RESET

HISTÓRIA

Fotos do protótipo da placa-mãe para o IBM PC!



HISTÓRIA

Em 1983, a IBM lança uma versão melhorada do IBM PC: o IBM PC/XT (*eXtended Technology*)

- 256 Kbytes de RAM (expansão até 640 kbytes)
- Capacidade para disco rígido de 10 MB
- 8 slots de expansão
- Clock 8 MHz (Turbo)



MODOS DE OPERAÇÃO

Modo Mínimo

O 8086/8088 gera os sinais de controle para as operações de memória e de E/S

Compatível com sistemas baseados no processador 8085

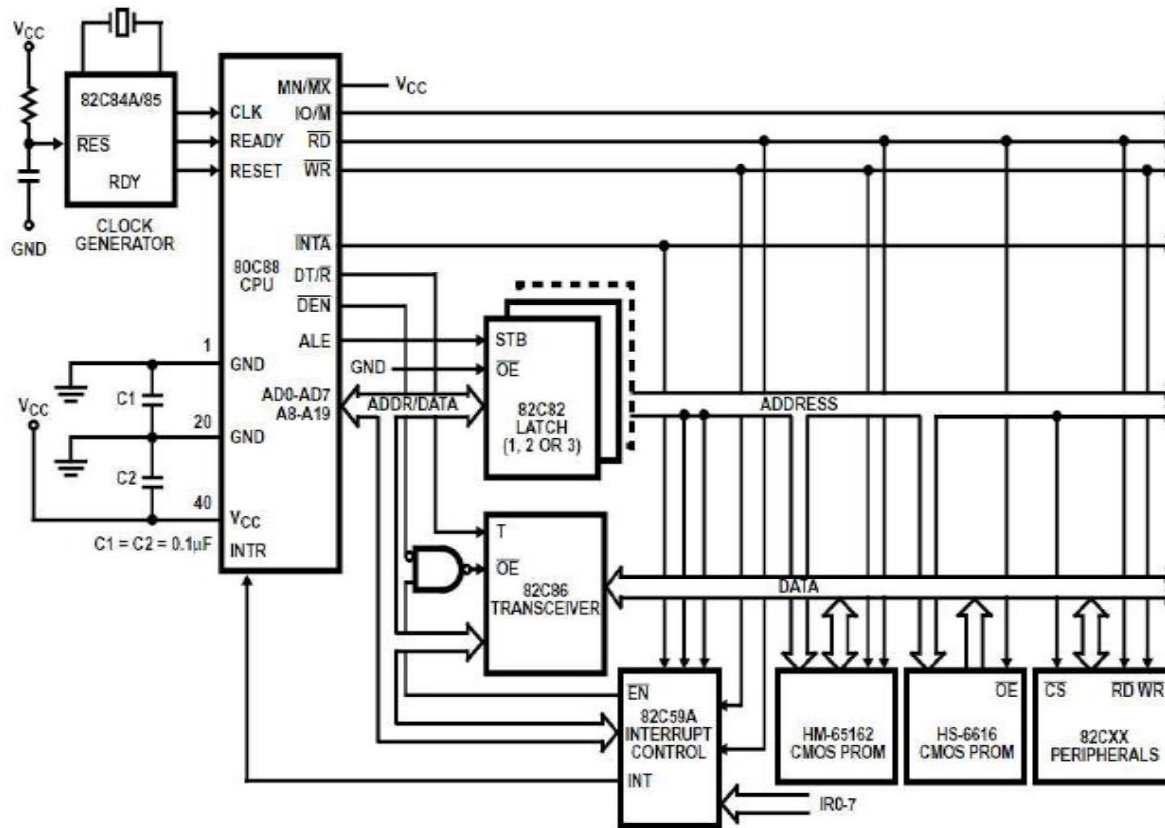
Modo Máximo

Necessita de um controlador de barramento (CI 8288) para gerar os sinais de controle das operações de memória e de E/S

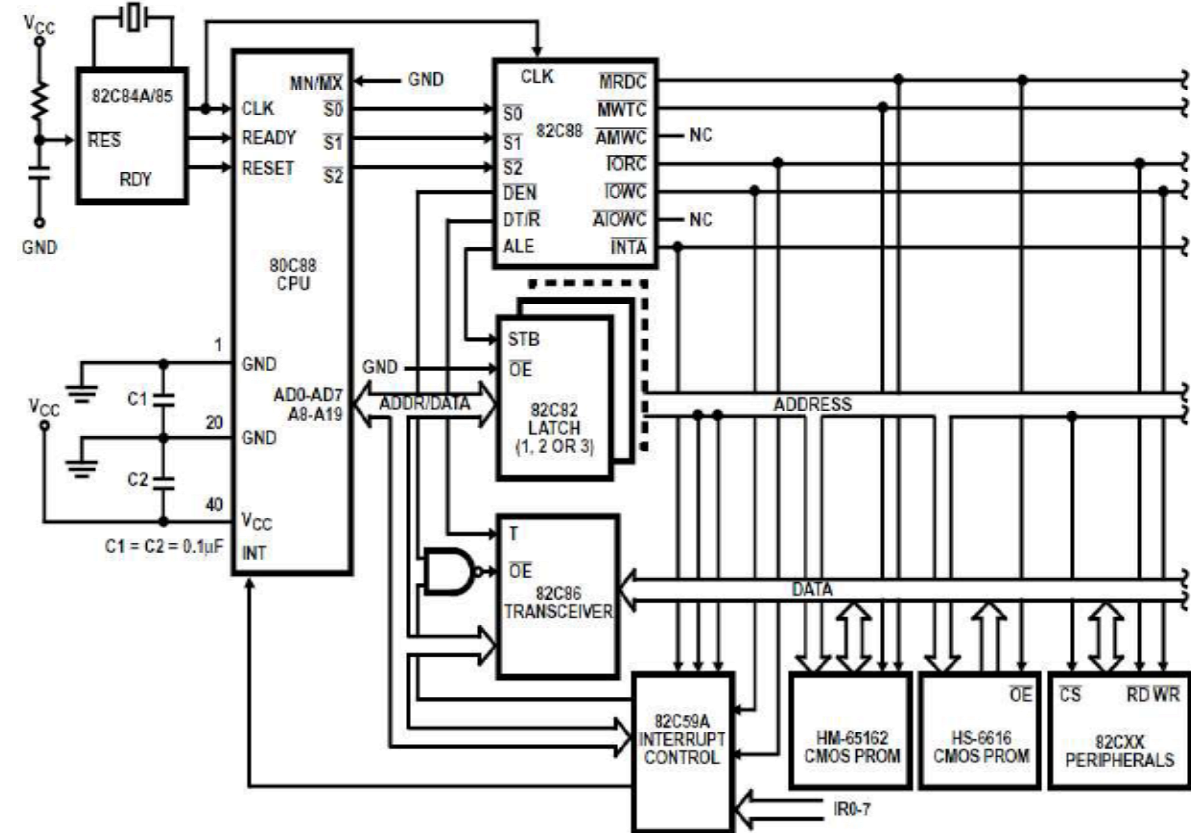
Permite o uso do coprocessador 8087 (ponto flutuante)

MODOS DE OPERAÇÃO

Modo Mínimo



Modo Máximo



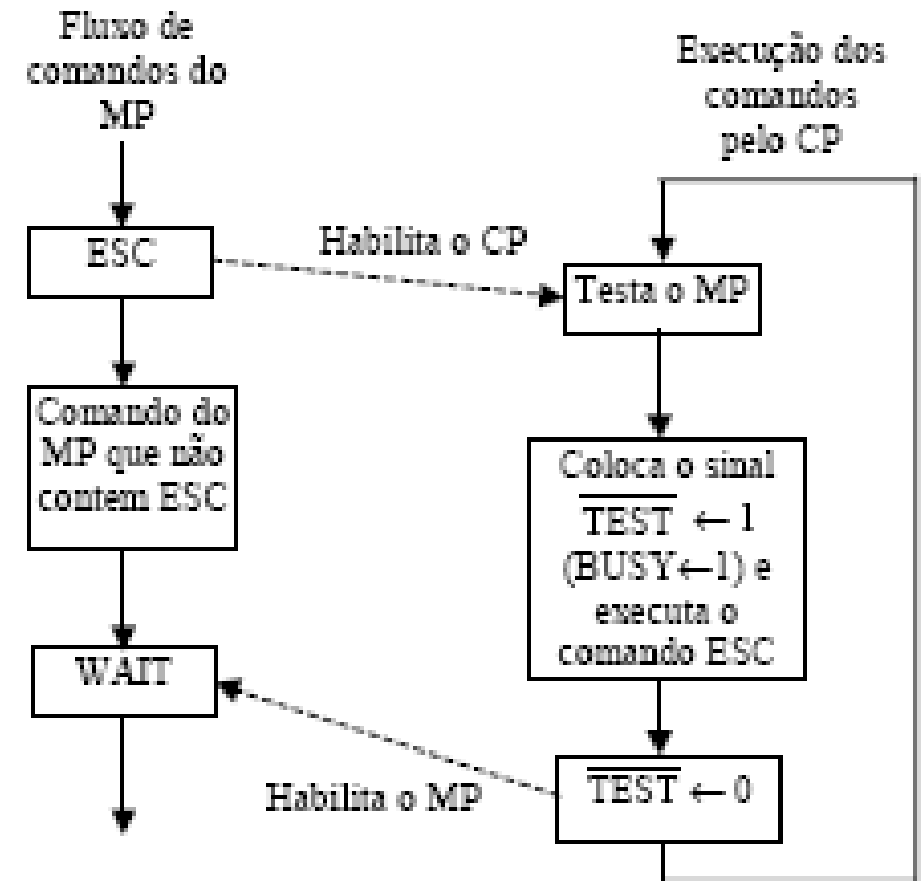
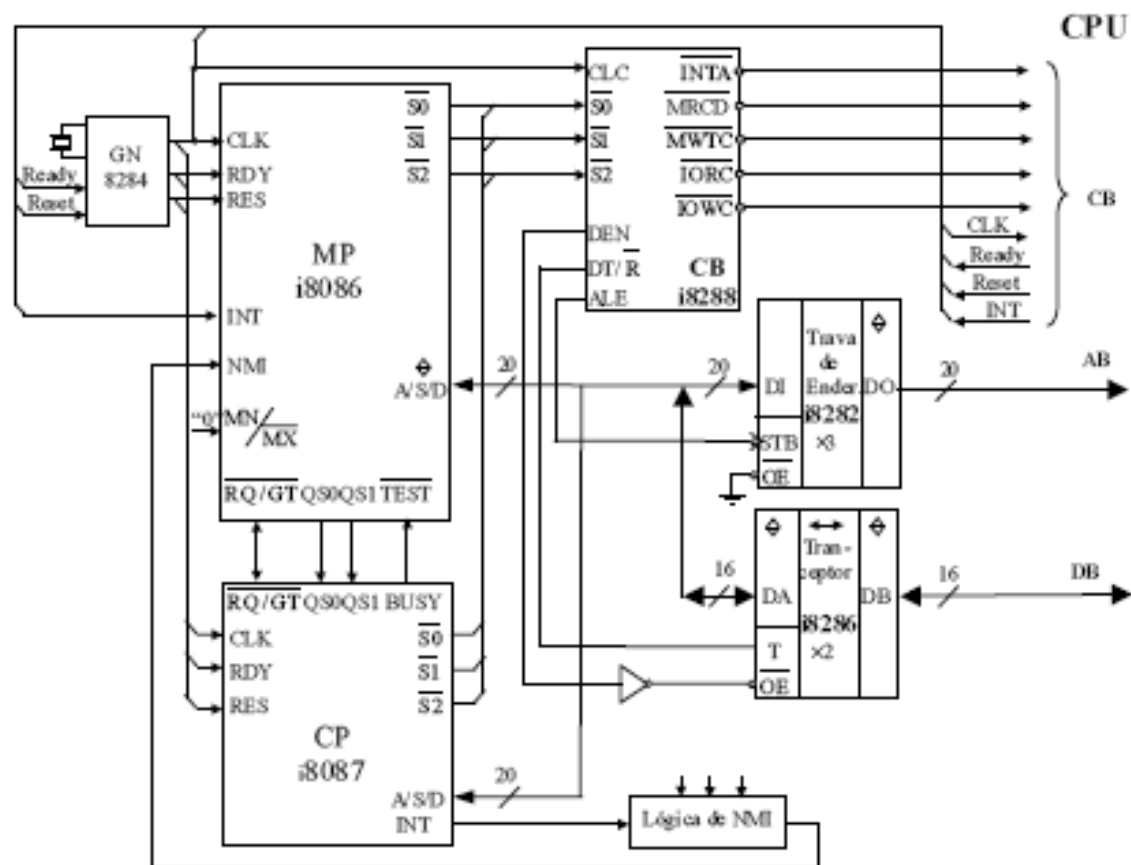
MODOS DE OPERAÇÃO: MÁXIMO

Coprocessador i8087 (1980)

- 45 K transistores, 50 kFLOPS (*Floating Operations per Second*)
- Adiciona 68 instruções numéricas para funções de inteiros de precisão estendida, ponto flutuante (PF), trigonométrica, logarítmica, e exponencial
 - Desempenho geral até 100 vezes do que o 8086 para instruções numéricas (soma de PF: $10.6 \mu s \times 1000 \mu s$)
- 8 registradores (propósito específico) de 80 bits
- 7 tipos de dados (16, 32, e 64 bits inteiros, 32 e 64 bits ponto flutuante, 18 dígitos BCD (*Binary Coded Decimal*))



I8086/8088: MODO MÁXIMO



ARQUITETURA DO PROCESSADOR

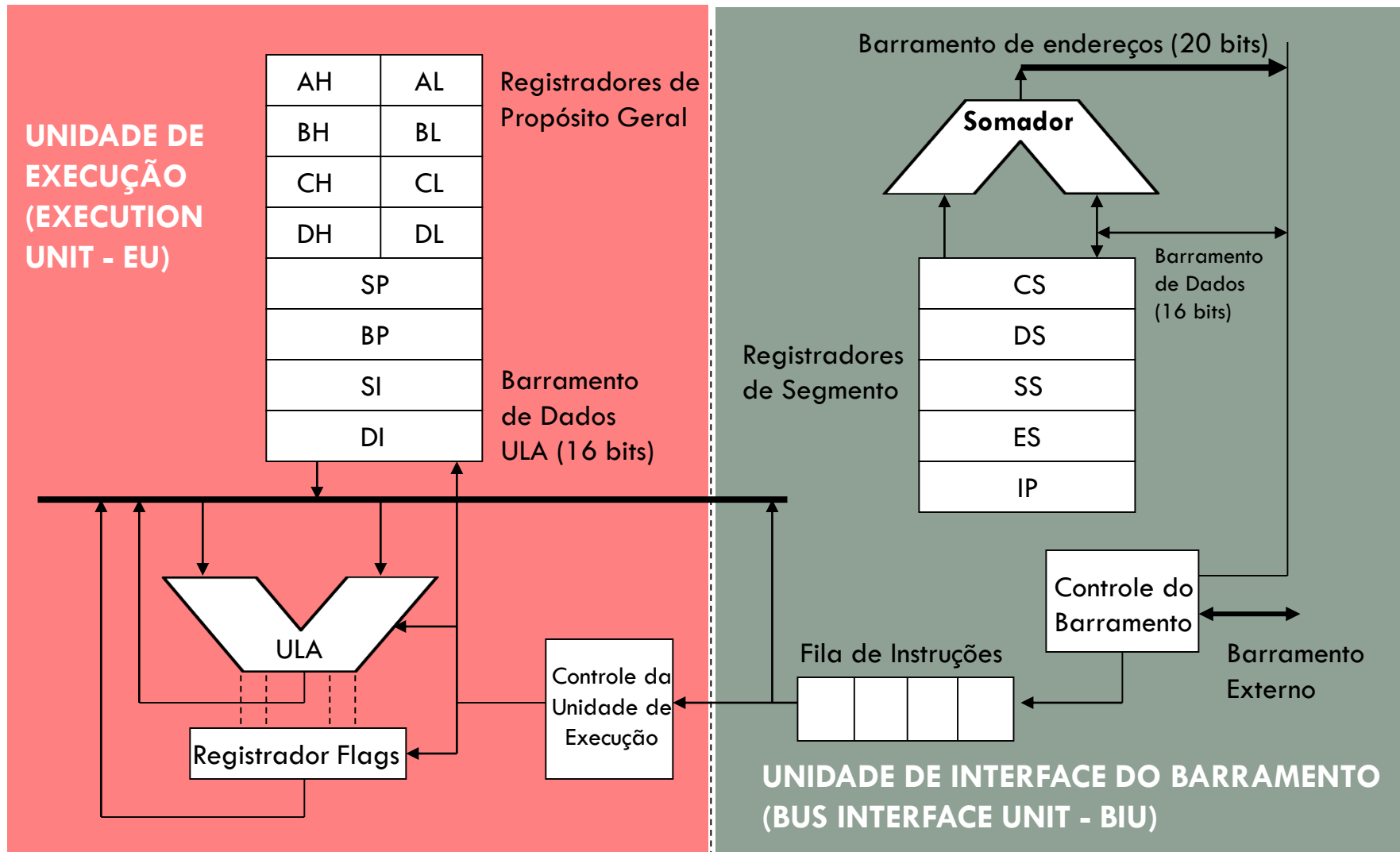
Microprocessadores executam um programa repetidamente passando pelos passos

1. Busca próxima instrução na memória
2. Leitura do operando da memória, se necessário
3. Execução da instrução
4. Escrita do resultado, se necessário

O 80886/8088 utiliza duas unidades independentes (permitir a sobreposição de busca e execução) de processamento para executar esses passos:

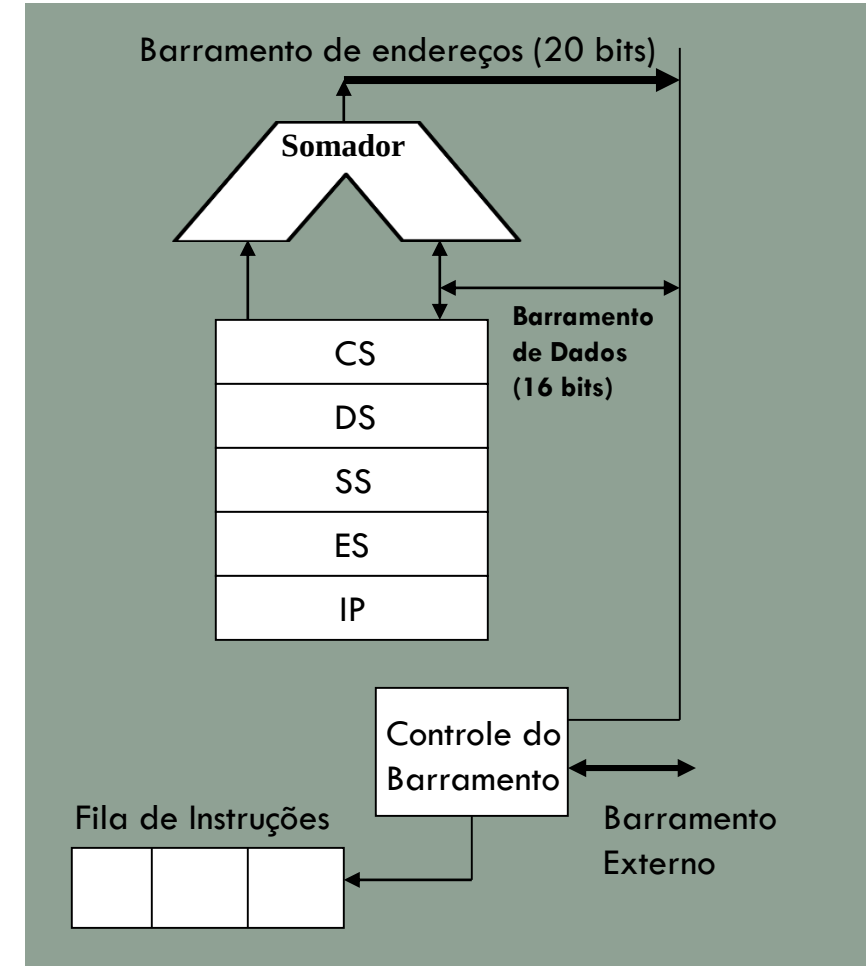
- Unidade de Execução
- Unidade de Interface do Barramento:

ARQUITETURA DO PROCESSADOR



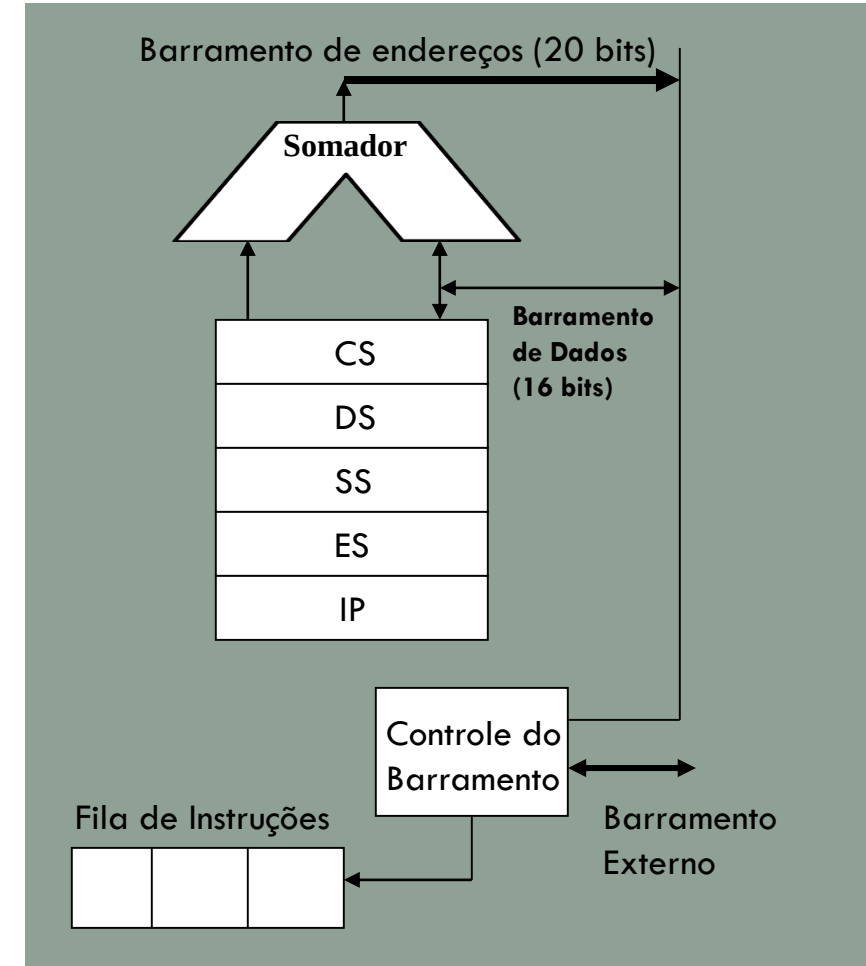
BUS INTERFACE UNIT: OPERAÇÕES

1. Forma o endereço físico do barramento
2. Carrega as instruções e dados da memória e de E/S
3. Forma a fila de instruções
4. Transfere os resultados para a memória e E/S
5. Troca informações com a Unidade de Execução



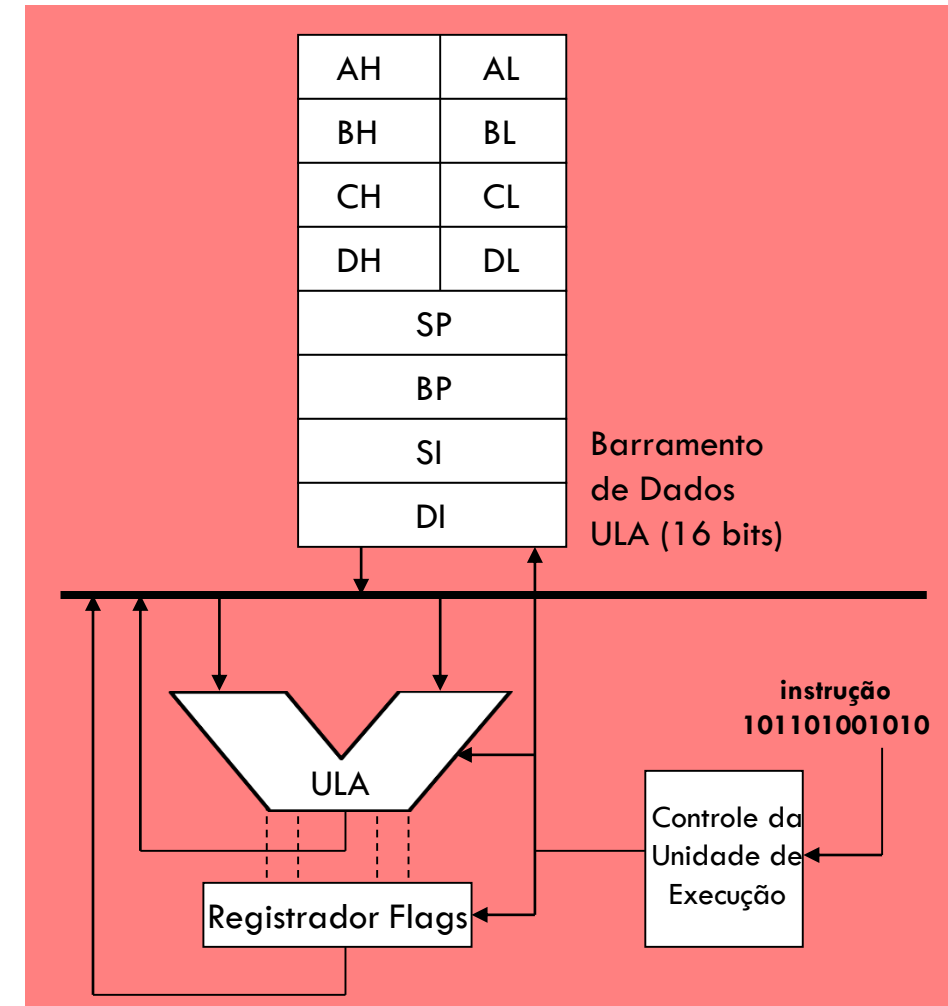
BUS INTERFACE UNIT: INFRAESTRUTURA

- 4 registradores de segmento de 16 bits (endereçamento da memória)
- 6 registradores de 8 bits (4 para o 8088) na fila (FIFO) de instruções
- 2 registradores internos de comunicação com EU
- 1 registrador de 16 bits que é o ponteiro de instruções (IP - *Instruction Pointer*)
- Somador para calcular 20 bits do endereço físico com base em 2 fontes (registradores de 16 bits cada)



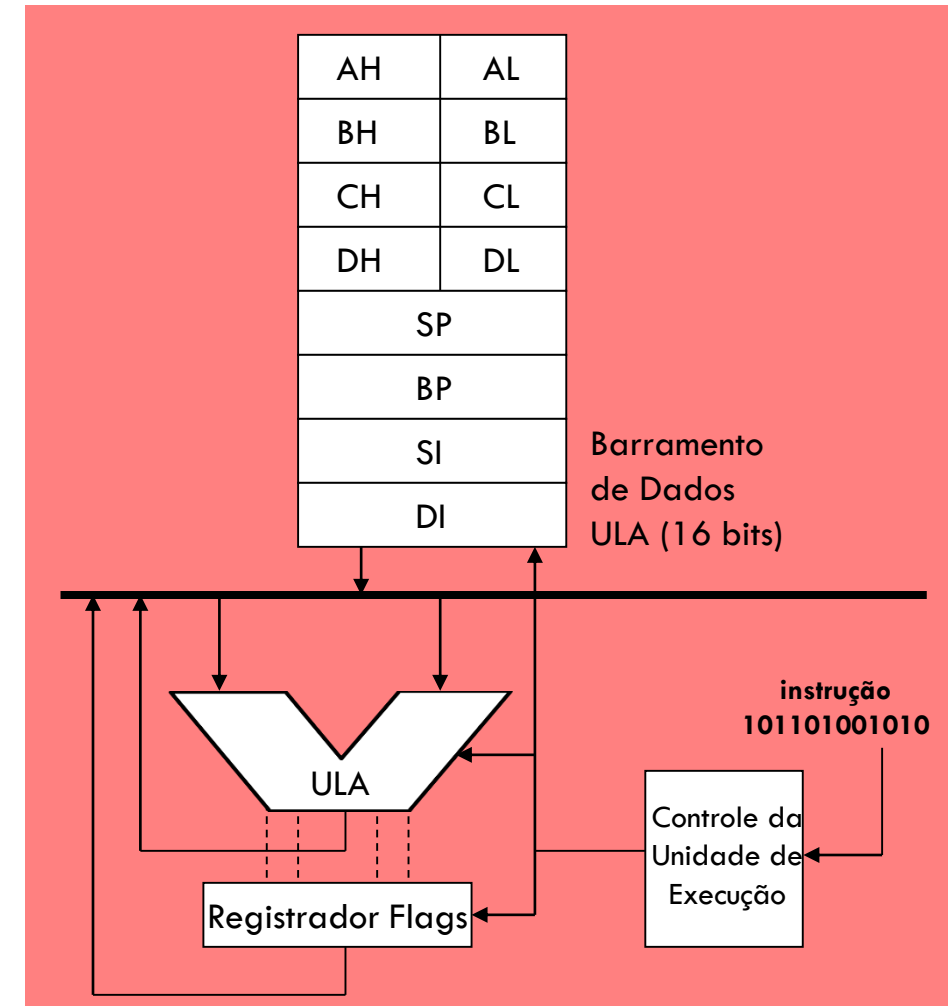
UNIDADE DE EXECUÇÃO: OPERAÇÕES

1. Busca uma instrução da fila de instruções
2. De acordo com a instrução, a lógica de controle da EU gera os sinais de controle (decodificação da instrução)
3. Dependendo do sinal de controle, a EU executa uma das seguintes operações
 - Uma operação aritmética
 - Uma operação lógica
 - Armazenar um dado em um registrador
 - Mover um dado de um registrador
 - Mudar o registrador flag



UNIDADE DE EXECUÇÃO: INFRAESTRUTURA

- Unidade Lógica e Aritmética (ULA) de 16 bits
- 3 registradores temporários (não acessíveis pelo programador)
- 1 registrador de status (FLAGS) de 16 bits
- 8 registradores de 16 bits de uso geral



UNIDADE DE EXECUÇÃO: REGISTRADORES

Registradores de Dados

- AX (AH+AL): acumulador
- BX (BH+BL): base para referenciar memória
- CX (CH+CL): contador
- DX (DH+DL): dados (multiplicação e divisão de 32 bits)

Registradores de Ponteiro e Índice

- SP: ponteiro de pilha (*Stack pointer*)
- BP: ponteiro de base (*Base pointer*)
- SI: índice de fonte (*Source Index*)
- DI: índice de destino (*Destination Index*)

	15	8	7	0
AX	AH		AL	
BX	BH		BL	
CX	CH		CL	
DX	DH		DL	
SP				
BP				
SI				
DI				

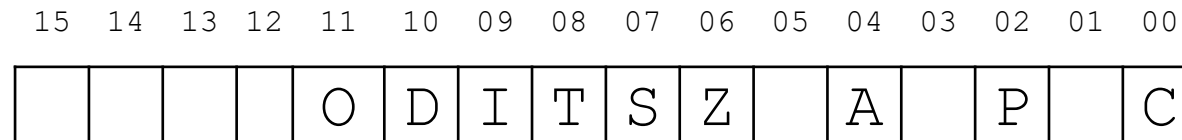
Alguns dos registradores são utilizados de forma implícita pelas instruções

UNIDADE DE EXECUÇÃO: REGISTRADORES

O registrador de status do processador (flags) tem por objetivo dar suporte às decisões dentro do programa e habilitar/desabilitar certas operações do processador

Tipos

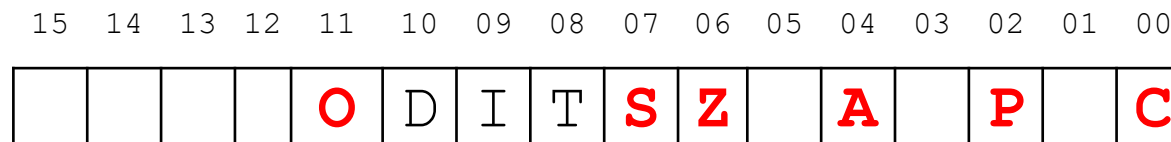
- Flags de status – resultado de uma operação aritmética ou lógica (ULA)
- Flags de controle – habilitar / desabilitar operações do processador



UNIDADE DE EXECUÇÃO: REGISTRADORES

Flags de Status

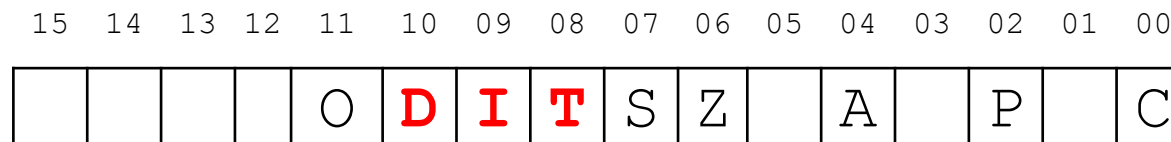
- **Carry (CF):** ligado se uma operação de adição resulta em um *carry*, ou se uma subtração resulta em um *borrow* ($CF = 1$)
- **Paridade (PF):** ligado quando um valor contém um número par de bits ligados ($PF = 1$ – paridade par)
- **Carry Auxiliar (AF):** semelhante ao CF, porém é usado para indicar um *carry* ou um *borrow* nos quatro bits menos significativos ($AF = 1$)
- **Zero (ZF):** ligado quando o resultado de uma operação é zero
- **Sinal (SF):** indica se um número é negativo ou positivo (bit de sinal do resultado é copiado para este flag)
- **Overflow (OF):** ligado quando resultado excede limites do tamanho de operandos



UNIDADE DE EXECUÇÃO: REGISTRADORES

Flags de Controle

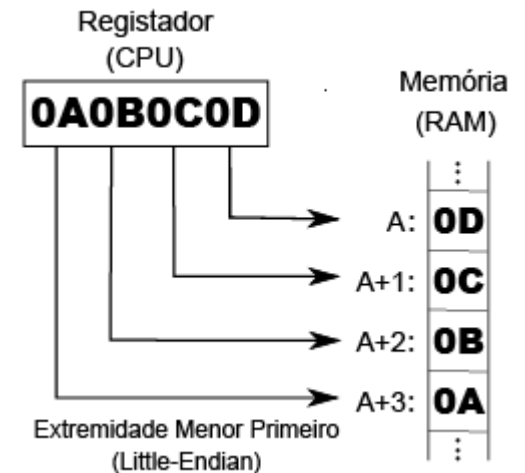
- **Trap (TF):** controla execução de comandos por passos (TF=1 faz o MP para após cada instrução)
- **Interrupção (IF):** define se as interrupções mascaráveis podem se executadas (IF=1) ou não (IF=0) permite interrupções (instruções STI e CLI)
- **Direção (DF):** determina a direção em que instruções de *strings* são processadas (em relação a SI e DI). Se DF = 0, direção é a partir do endereço mais significativo.



ORGANIZAÇÃO DA MEMÓRIA

O 8086/8088 possui um espaço de endereçamento de 1MB (00000H a FFFFFH), isto é, o barramento de endereços possui uma largura de 20 bits

Seguindo a convenção da Intel, as palavras são armazenadas na memória utilizando a ordem *little endian*, isto é, os bytes são armazenadas a partir de menos significativo ao mais significativo



ORGANIZAÇÃO DA MEMÓRIA: 8088

No 8088, o espaço de endereçamento de memória é logicamente organizado como um vetor linear de 1M bytes

Como o 8088 possui barramento de dados de 8 bits, a memória consiste de um único banco

- Acesso a um byte no 8088 (1 acesso)

0000 AA um byte no endereço 0000

- Acesso a palavra (16 bits) no 8088 (2 acessos)

0000 BB byte menos significativo no endereço 0000

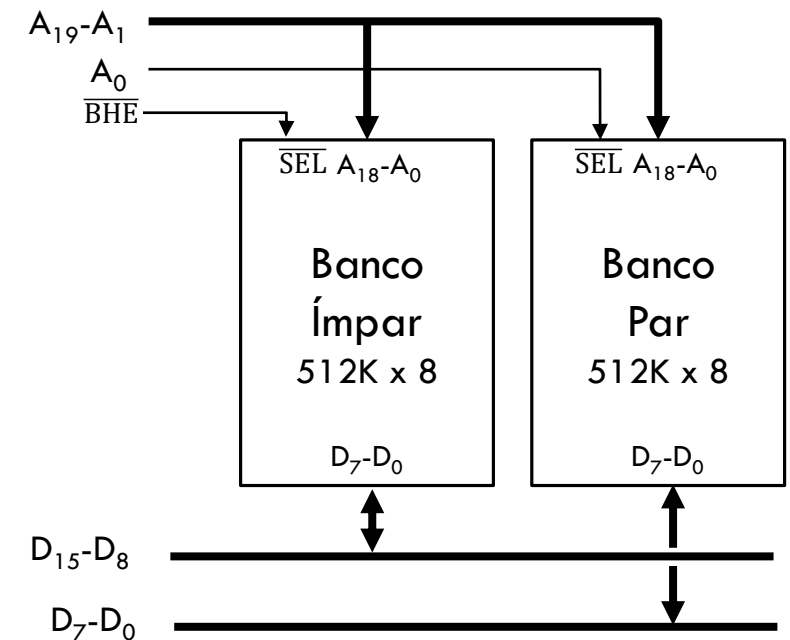
0001 AA byte mais significativo no endereço 0001

ORGANIZAÇÃO DA MEMÓRIA: 8086

No 8086, 2 bytes consecutivos podem ser acessados simultaneamente como se fosse uma palavra de 16 bits (alinhamento da palavra)

O espaço de endereçamento da memória é fisicamente dividido em dois bancos de 512K bytes cada

- Um banco é associado com a parte menos significativa do barramento de dados (D_7-D_0) e o outro banco é associado com a parte mais significativa ($D_{15}-D_8$)
- A_0 é utilizado para selecionar o banco par, enquanto que sinal $\overline{BHE}$ é utilizado para selecionar o banco ímpar



ORGANIZAÇÃO DA MEMÓRIA: 8086

Como o 8086 possui barramento de dados de 16 bits, o acesso a um único **byte** descarta o **byte supérfluo**, independente do endereço em que estiver

Quando o acesso for a uma palavra em endereço par (byte menos significativo em banco par e mais significativo em banco ímpar), a palavra é dita estar **alinhada**, pois pode ser acessada com uma única operação

- A seguinte operação é realizada em um único acesso

0000 BB byte menos significativo no endereço 0000

0001 AA byte mais significativo no endereço 0001

ORGANIZAÇÃO DA MEMÓRIA: 8086

Quando o acesso for a uma palavra em endereço ímpar (byte menos significativo em banco **ímpar** e mais significativo em banco **par**), a palavra é dita estar **não alinhada**, e deve ser acessada em dois ciclos

- A seguinte operação é realizada em 2 acessos

0000 nn	byte par anterior lido e descartado
0001 BB	byte de baixa ordem acessado
0002 AA	byte de alta ordem acessado
0003 nn	byte ímpar seguinte lido e descartado

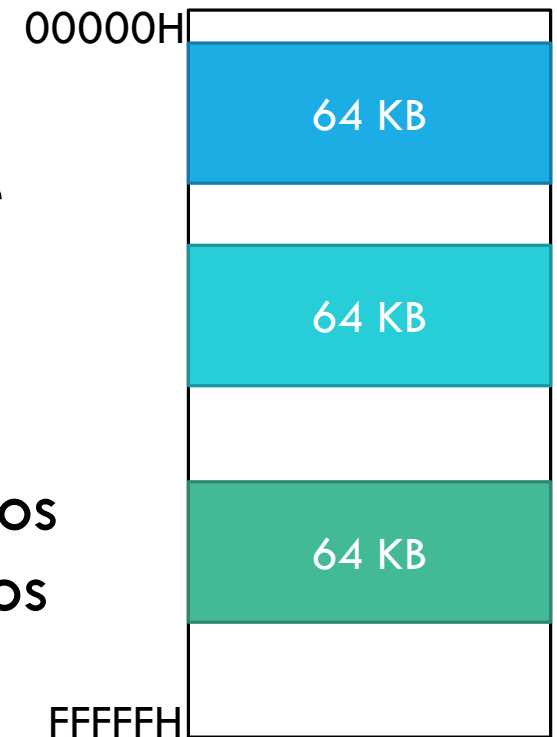
	Banco Par	Banco Ímpar
2	AA	nn
0	nn	BB

ORGANIZAÇÃO DA MEMÓRIA: SEGMENTAÇÃO

Como os registradores possuem tamanho de 16 bits, o endereçamento de 20 é solucionado através da segmentação da memória

O 8086/80888 “enxerga” o espaço de memória como um grupo de segmentos de até 64 KB (registradores de 16 bits), definidos pela aplicação

Segmentos são locais de memória contíguas, posicionados em diferentes locais da memória e independentes um dos outros



ORGANIZAÇÃO DA MEMÓRIA: SEGMENTAÇÃO

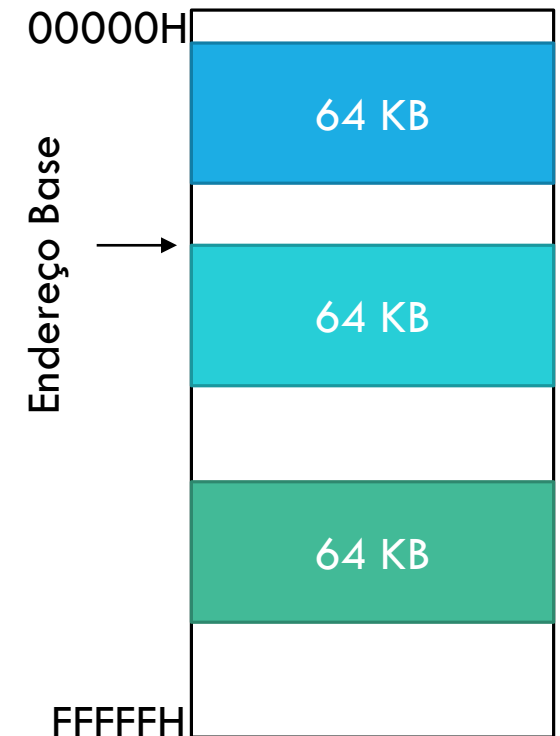
Cada endereço é posicionado utilizando um **endereço base**

- A base do segmento é armazenada em um registrador de 16 bits

O endereço físico (20 bits) do segmento é obtido **deslocando o valor do registrador de 16 bits 4 bits à esquerda**

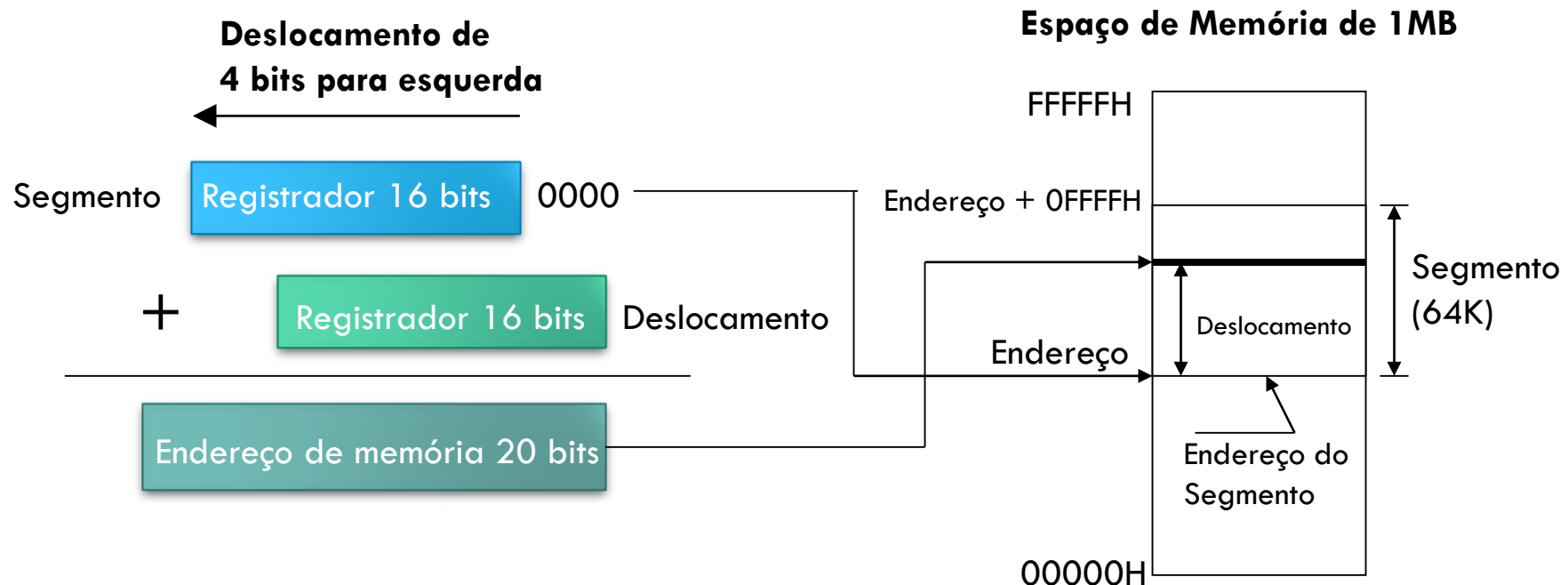
- Equivalente a multiplicar por 16 (10H)

Registrador	Endereço base do segmento
0000H	00000H (primeiro segmento)
0001H	00010H
1000H	10000H
FFFFH	FFFF0H (Último segmento válido)



ORGANIZAÇÃO DA MEMÓRIA: SEGMENTAÇÃO

Uma posição de memória é definida através do endereço base (16 bits) do segmento e um deslocamento (16 bits)



○ processador trabalha com endereços lógicos

ORGANIZAÇÃO DA MEMÓRIA: SEGMENTAÇÃO

Como o endereço base do segmento é obtido através de um deslocamento de um registrador, os segmentos podem ser adjacentes, disjuntos e parcialmente sobrepostos

- Permite utilizar melhor a memória de tal forma que cada segmento ocupe exatamente o que precisa
- Não há garantia de que o acesso de um segmento sobreposto seja realizado dentro do respectivo segmento (pode estender até o segmento sobreposto)
- Um endereço físico de memória pode ser mapeado em 1 ou mais segmentos lógicos

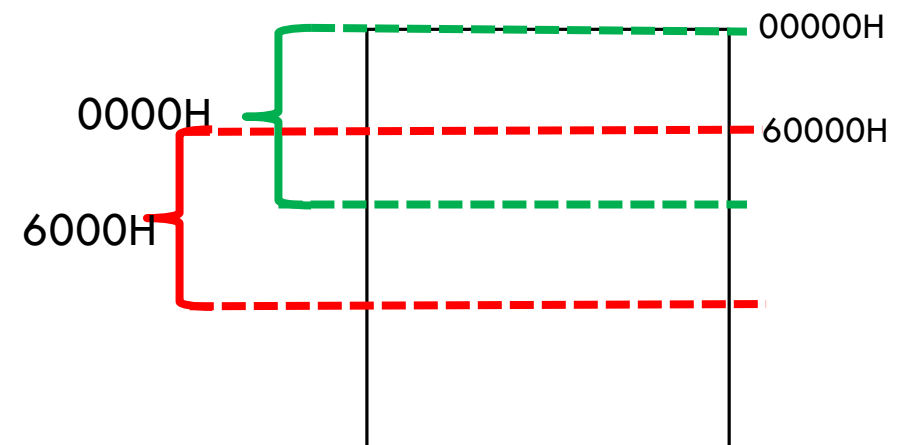
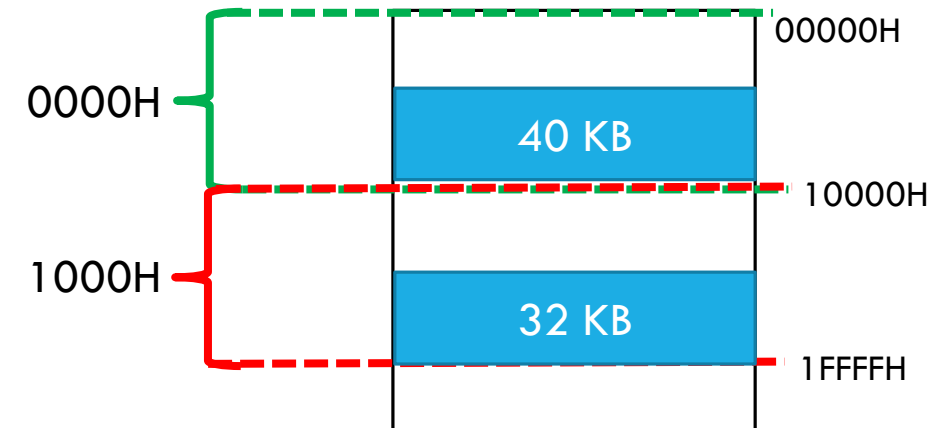
ORGANIZAÇÃO DA MEMÓRIA: SEGMENTAÇÃO

Tamanho Segmento	Registrador	Memória
24KB	0000H	00000H
32KB	1000H	10000H

Desperdício de 72 KB

24KB	0000H	00000H
32KB	6000H	06000H

Sem desperdício por conta da sobreposição

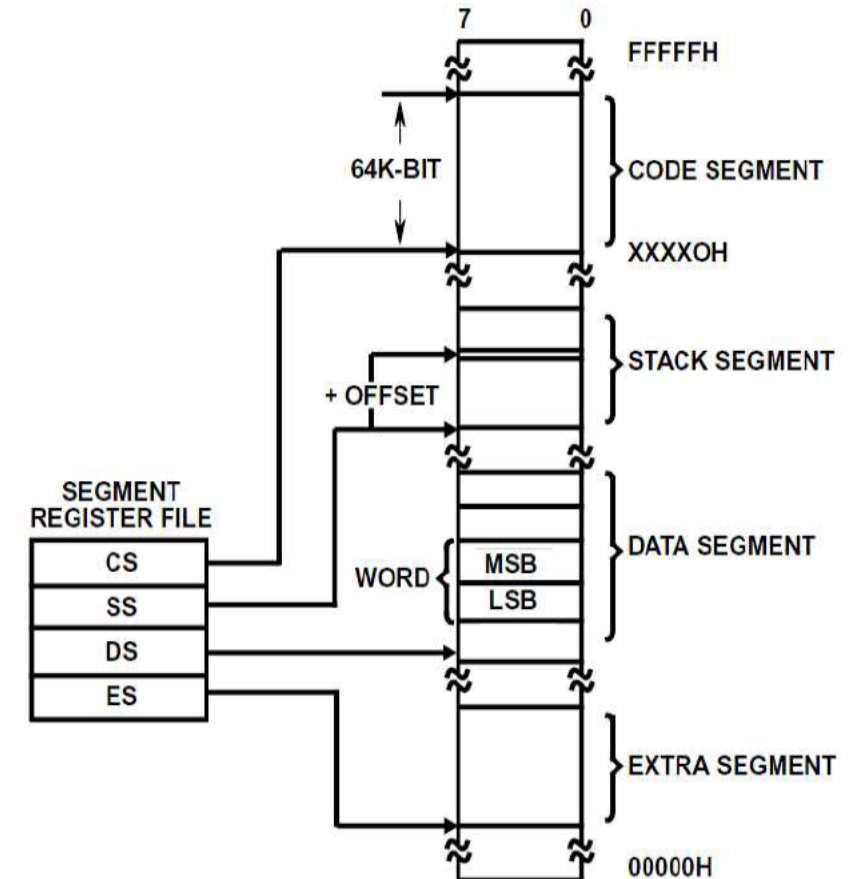


ORGANIZAÇÃO DA MEMÓRIA: SEGMENTAÇÃO

A arquitetura possui **4 registradores de segmento** de 16 bits cada

- CS (**Code Segment**): instruções a serem executadas pelo MP
- SS (**Stack Segment**): uso reservado para a pilha ou stack (LIFO)
- DS (**Data Segment**): segmento de dados
- ES (**Extra Segment**): segmento de dados auxiliares

Cada programa pode definir e utilizar os segmentos diferentemente



ORGANIZAÇÃO DA MEMÓRIA: SEGMENTAÇÃO

O endereço do segmento é armazenado em registradores de segmento

O deslocamento é resultado da combinação de registradores de ponteiro, *instruction pointer* e valores imediatos (constantes)

Um endereço é representado pelo segmento e seu deslocamento – SEGMENTO:DESLOCAMENTO

A BIU realiza o cálculo do endereço físico para acessar a memória

Endereço do Segmento	0000
+	Deslocamento
Endereço de Memória	

CS	3	4	8	A	0
IP +		4	2	1	4
Endereço de Instrução	3	8	A	B	4

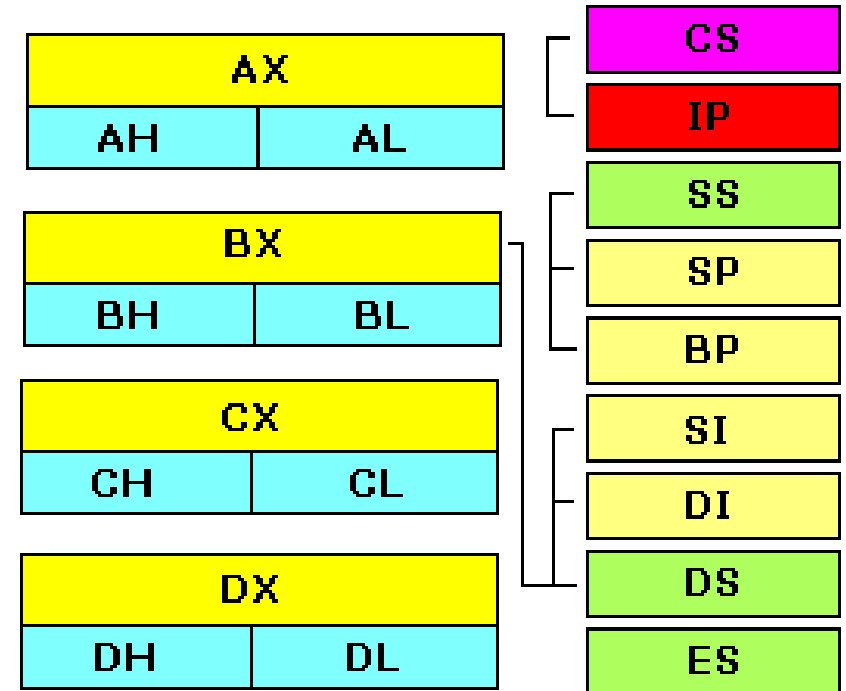
SS	5	0	0	0	0
SP +		F	F	E	0
Endereço de Pilha	5	F	F	E	0

DS	1	2	3	4	0
DI +		0	0	2	2
Endereço de Dado	1	2	3	6	2

ORGANIZAÇÃO DA MEMÓRIA: SEGMENTAÇÃO

Registradores (como endereços lógicos) envolvidos na formação do endereço físico

- CS e IP para endereçar código
- SS e (SP, BP) para endereçar a pilha
- DS (e/ou ES) e BX, SI e DI para endereçar dados (DI é relacionado ao ES quando parte endereço de um operando de instrução de string é o destino)



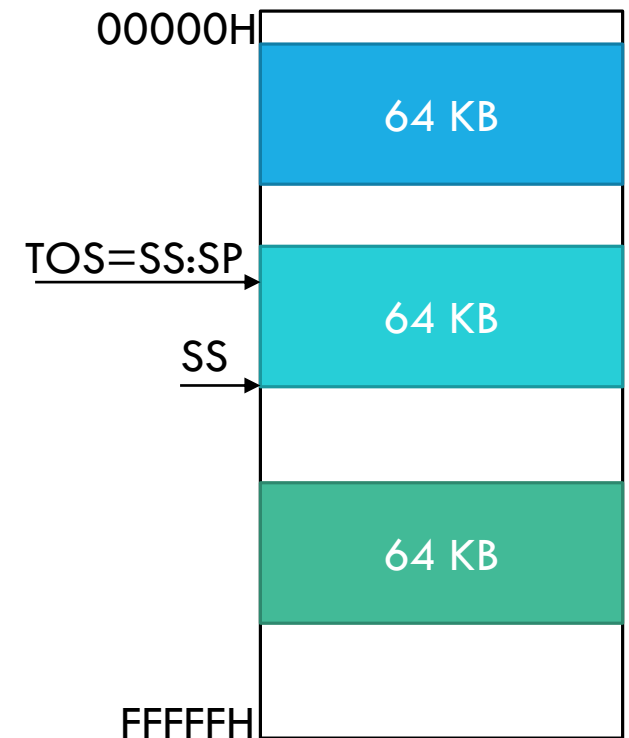
ORGANIZAÇÃO DA MEMÓRIA: PILHA (STACK)

O segmento de pilha é definido pelo registrador SS (*Stack Segment*) e tem por objetivo armazenar dados temporários (LIFO), endereço de retorno de rotinas e variáveis locais

O deslocamento é definido pelo registrador SP (*Stack Pointer*)

- Topo da Pilha (*Top of Stack* -- TOS) é definido por SS:SP

Endereços variam do mais alto para o mais baixo (endereço decrescem)



ORGANIZAÇÃO DA MEMÓRIA: PILHA (STACK)

Duas instruções manipulam a pilha

- **push (empilha)**

1. Atualiza o deslocamento dentro da pilha: $SP \leftarrow SP - 2$
2. Escreve uma palavra de dados na pilha nos endereços $SS:SP$ e $SS:SP+1$

- **pop (desempilha)**

1. Lê uma palavra da pilha dos endereços $SS:SP$ e $SS:SP+1$
2. Atualiza o deslocamento dentro da pilha $SP \leftarrow SP + 2$

```
push 1234H  
push 5678H  
pop AX
```



End	Memória(1 byte)
X-6	
X-5	
X-4	78H
X-3	56H
X-2	34H
X-1	12H
X	Início da Pilha

MAPAS DE MEMÓRIA E I/O

Mapa da Memória

Endereço:		Endereço:		
00000H	64K RAM stand	00000H	Tabela de vetores de interrupção	Tabela de vetores de interrupção
...		...	BIOS (0H...1FH)	
0FFFFH	192K RAM da placa mãe	0007FH	Tabela de interrupções DOS	
10000H		00080H	Reserva	
...		...	Tabela de valor do vetor do usuário	
3FFFFH		000FFH	Vetor de interrupção BASIC	
40000H	384K expansão da RAM	00100H	Dados para BIOS	
...	128K RAM placa de vídeo	...	Dados para DOS e Basic	
9FFFFH	ROM (216K) de expansão	00180H	Área acessível pelo usuário	
A0000H		...		
BFFFFH	Interpretador BASIC	00200H		
C0000H	ROM (32K)	...		
...	ROM (8K)	00400H		
F5FFFH	BIOS	...		
F6000H		00500H		
...		...		
FDFFFH		00600H		
FE000H		...		
...		0FFFFH		
FFFFFH				

Mapa de I/O

Endereço:		Endereço:	
000H	Controlador de DMA i8237	080H	Paginação de DMA
...		...	
083H		083H	
...	PCI i8259A	...	Máscaras do NMI
020H		0A0H	
021H		...	
...	TIMER i8253	0AFH	RESERVA
040H		0C0H	
...		...	
043H	PPA i8255A	0FFH	
...		...	
060H		...	
...			
063H			
...			

INTERRUPÇÕES

Os sinais de interrupção dos dispositivos externos podem chegar pelo pino INTR (através da PIC 8259) ou pelo pino NMI (*non-maskable interrupt*)

- Iminente perda de energia, detecção de erro de memória ou erro de paridade de barramento são alguns dos eventos que gera uma NMI

Cada interrupção é identificada por código de tipo de 8 bits, permitindo que o 8086/8088 trabalhe com até 256 tipos de interrupções

INTERRUPÇÕES

O código identifica uma entrada na tabela de 256 entradas, que contém ponteiros de endereço para as rotinas de interrupção, tabela de vetores de interrupção

- “vetor” representa um ponteiro

Cada entrada nesta tabela consiste em dois valores de endereço de 16 bits (quatro bytes) que são carregados nos registradores de segmento de código (CS) e ponteiro de instrução (IP) como o endereço da rotina de interrupção

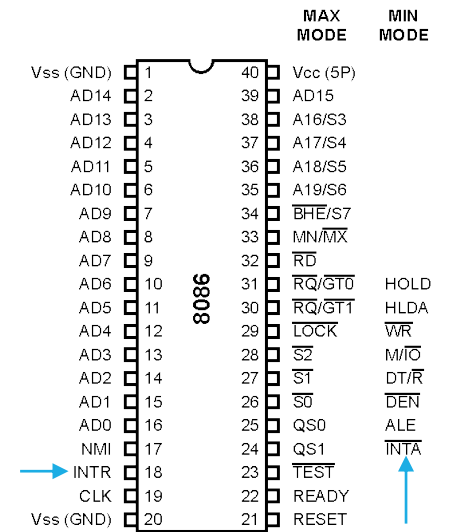
A tabela reside em locais absolutos do endereço 0 até 3FFH (1024 bytes)

INTERRUPÇÕES

Tipo n de int. (Hex)	IRQ _i	Tipo de interrupção (do 8259A)								Endereço da mem. (Tabela de vetores)	Periféricos que causam a interrupção
		D7	D6	D5	D4	D3	D2	D1	D0		
8	0	0	0	0	0	1	0	0	0	20H	Relógio
9	1	0	0	0	0	1	0	0	1	24H	Teclado
A	2	0	0	0	0	1	0	1	0	28H	Reservado
B	3	0	0	0	0	1	0	1	1	2CH	Com. Serial 1
C	4	0	0	0	0	1	1	0	0	30H	Com. Serial 2
D	5	0	0	0	0	1	1	0	1	34H	Disco Rígido
E	6	0	0	0	0	1	1	1	0	38H	Disco Flexível
F	7	0	0	0	0	1	1	1	1	3CH	Impressora

INTERRUPÇÕES: PROCESSO

1. MP i8088/8086 recebe o pedido de interrupção pelo pino **INT** do **PIC i8259A**
2. O MP após o término da instrução em execução, responde pelo sinal **INTA'**
 - 1º sinal **INTA' = 0** (T2, T3 de CLK)
 - 2º sinal **INTA = 0** (T2, T3 de CLK do próximo ciclo)
3. Recebendo este sinal, o PIC 8259A coloca no barramento de dados (*data bus* – DB) o **byte** de tipo de interrupção ***n*** (0 a 255) associado com o dispositivo requisitante do serviço
4. Flags, CS e IP do programa interrompido são transferidos à pilha (nesta ordem)
5. $IF, TF \leftarrow 0$ automaticamente, proibindo a interrupção da outra fonte



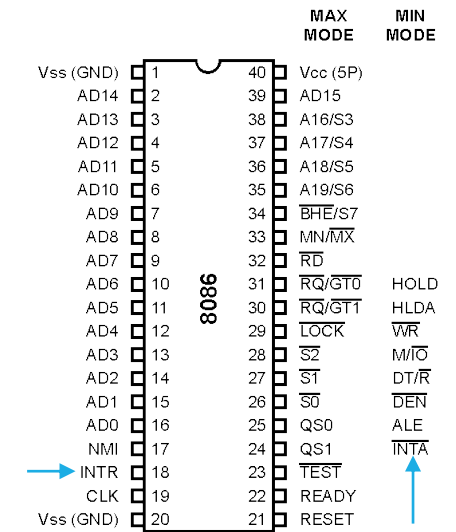
INTERRUPÇÕES

6. O endereço da rotina de tratamento da interrupção é obtido na tabela de vetores de interrupção na memória, localizada no endereço 0 da memória, com base no tipo da interrupção, n . O endereço é inicializado nos registradores IP e CS.

$$IP \leftarrow \text{MEM}[4*n]$$

$$CS \leftarrow \text{MEM}[4*n+2]$$

7. MP executa a sub-rotina de atendimento desta interrupção que começa pelo endereço novo CS:IP
- Para liberar as interrupções do nível superior, a sub-rotina deve conter o comando `STI` que faz $IF \leftarrow 1$
 - A sub-rotina de tratamento de interrupção deve ser finalizada pelo comando `IRET` que recupera IP, CS e Flags da pilha
8. Retorno ao programa interrompido
- MP começa executar o próximo comando deste programa de acordo com IP, CS e “Flags” recuperados da pilha



INTERRUPÇÕES

Tipo <i>n</i> de Interrup.	Função	Tipo <i>n</i> de Interrup.	Função
0	Divisão por 0	14	I/O de RS232
1	Processo de Single Step	15	I/O gravador cassete
2	Interrupção não mascarável NMI	16	I/O de teclado
3	Interrupção de Break Point	17	I/O de impressora
4	Overflow (estouro)	18	ROM BASIC
5	Impressão do vídeo (Print_Screen)	19	Reboot
6,7	Reservadas	1A	Relógio do dia
8	Relógio	1B	Falha do teclado
9	Teclado	1C	Interrupção do Timer
A	Reservada	1D	Ponteiro para inicialização do vídeo
B,C	Comunicação serial	1F	Ponteiro para inic. do gerador gráfico
D	Disco rígido	20 – 3F	Usado pelo DOS
E	Disco flexível	40 – 5F	Reservado
F	Impressora	60 - 67	Reservado para o usuário
10	I/O de vídeo	68 – 7F	Não usado
11	Checagem de hardware	80 - 85	Reservado para o BASIC
12	Checagem de memória	86 – F0	Usado pelo interpretador Basic
13	I/O de disco flexível	F1 - FF	Não usado